

Jalaram Feeds ERP

A full-scale ERP for an animal-feed manufacturer, migrated from Google Apps Script to the Cloudflare edge.

Six Workers. Eight D1 databases. Twenty-two thousand lines of TypeScript-free JavaScript. A single-login "Gate" pattern. Zero downtime migration. This is how it was designed, built, and shipped.

CLOUDFLARE WORKERS

D1 (SQLITE AT THE EDGE)

SERVICE BINDINGS

WORKERS AI

BROWSER RENDERING

R2 OBJECT STORAGE
AUTHORED BY

Abhinav Randai

github.com/Abhinavrandai

linkedin.com/in/abhinavrandai

abhinavrandai@gmail.com

6

WORKERS

8

D1 DBS

39

TEST FILES

22k

LINES

Contents

1. Executive Summary	3
2. Business Context & Migration Story	4
2.1 Why migrate at all	4
2.2 The gas-shim.js proxy pattern	4
2.3 Before and after	4
3. System Architecture: The Gate Pattern	5
3.1 Why a single entry point	5
3.2 Session handoff to module workers	5
3.3 Worker inventory	5
4. Authentication & Authorization	7
4.1 Password hashing	7
4.2 Per-module RBAC	7
4.3 Mobile Link — B110	7
5. RPC Pattern: One URL, Many Functions	8
5.1 The RPC table doubles as the access-control surface	8
5.2 Why not REST or GraphQL	8
6. Database Topology: 8 D1 Databases	9
6.1 The stock-sync flow	9
7. Real-World Features Deep Dive	10
7.1 WhatsApp Business integration with PDF fallback (B166)	10
7.2 Sahayak AI chat assistant (Workers AI)	10
7.3 Day End report with Browser Rendering (B128)	10
7.4 Mobile Link for laboratory (B110)	11
8. Engineering Rigor	12
8.1 Test-driven development as a working habit	12
8.2 The "lazy senior dev" philosophy	12

8.3 B-number changelog convention 12

8.4 Coding conventions 12

9. Deployment & Operations14

9.1 Deploy checklist 14

9.2 Secrets management 14

9.3 Parked work 14

10. Lessons Learned & What I Would Do Differently15

10.1 What worked 15

10.2 What I would do differently 15

11. About the Author & Contact16

1. Executive Summary

Jalaram Feeds ERP is a complete enterprise resource planning system for an animal-feed manufacturing business. It manages the full operational lifecycle of the company: sales orders, dispatch, freight, raw material purchase, goods received notes (GRN), production batches, labour payroll, and laboratory test reports. The system is in active daily use across multiple plants and handles real business data.

The entire system runs on **Cloudflare Workers** at the network edge, backed by eight **Cloudflare D1** SQLite databases and an **R2** object bucket for help-desk attachments. The architecture follows a deliberately strict **"Gate" pattern**: a single public-facing Worker (jalaram-gate) handles all authentication, routing, and dashboard aggregation, while five private Workers (sales, purchase, production, labour, lab) are reachable only through Cloudflare service bindings. No Worker except the Gate is exposed to the public internet.

The codebase spans roughly **22,000 lines of vanilla JavaScript** (no TypeScript, no framework) and is shipped with **39 test files** covering the most critical paths. The migration from the previous Google Apps Script + Sheets stack was completed with **zero data loss and near-zero UI change** for end users, achieved by a thin proxy layer (gas-shim.js) that translates legacy google.script.run calls into Worker `/rpc/*` endpoints.

HEADLINE METRICS

6 Cloudflare Workers · **8** D1 databases · **1** R2 bucket · **9** cron jobs · **22k** lines of code · **39** test files · **~150** RPC endpoints · zero-downtime migration from Google Apps Script.

2. Business Context & Migration Story

2.1 Why migrate at all

The business had been running on Google Apps Script web apps bound to Google Sheets as the data store. That stack was simple to build on, but it had reached its ceiling. Sheet reads were slow under concurrent users; the script runtime imposed hard 6-minute execution limits; version control was awkward; and the frontends were tightly coupled to Apps Script's `google.script.run` RPC contract, which made every refactor expensive. New features were getting harder to land, and the business was actively asking for more.

The mandate for the rewrite was uncompromising: **zero data loss, zero downtime, and no retraining for end users**. The owner was clear that the shop-floor staff would not learn a new UI; the screens could change shape under the hood, but the buttons and tabs they pressed daily had to keep working exactly as before. That constraint dictated almost every architectural decision that followed.

2.2 The gas-shim.js proxy pattern

The key trick that made the migration painless was a tiny browser-side shim called `gas-shim.js`. It re-implements the Apps Script `google.script.run` API surface and transparently proxies every call to the new Worker's `/rpc/<functionName>` endpoint. From the perspective of the legacy frontends, nothing had changed. They still called `google.script.run.withSuccessHandler(fn).myBackendFunction(a, b)` — except that the call now hit a Cloudflare Worker instead of an Apps Script container. The backend functions returned the same shape of data, so the UI never noticed.

This meant the migration could proceed **module by module, screen by screen**, without ever having to flag-day the whole application. Sales was migrated first, then Purchase, then Production, while Gate (the new entry point) was built last as the unified front door. The lab and labour modules were entirely new builds on the new stack, never having existed on Apps Script.

2.3 Before and after

Aspect	Before (Apps Script)	After (Cloudflare Workers)
Compute	Apps Script runtime (6-min limit)	Workers (no hard limit, 50ms CPU)
Data store	Google Sheets (slow under load)	8 D1 SQLite databases (indexed)
Frontend RPC	<code>google.script.run</code>	<code>/rpc/*</code> over HTTPS (via <code>gas-shim.js</code>)
Auth	Per-file Google SSO	PBKDF2 + D1 sessions (8-hour TTL)
Public exposure	All apps public	Only Gate public; 5 modules private
Cron	Time-driven triggers	Cloudflare Cron Triggers (9 jobs)
Version control	Awkward per-file	Git monorepo, single main branch
Tests	None	39 test files (plain node)

MIGRATION TAKEAWAY

The cheapest way to migrate a legacy line-of-business app is rarely a rewrite. A 200-line shim that preserves the old RPC contract lets you swap backends module by module while users keep clicking the same buttons. The hard work is in the data layer — not the UI.

3. System Architecture: The Gate Pattern

The defining architectural decision of this project is the **Gate pattern**. Cloudflare Workers can be exposed publicly or kept private (the `workers_dev: false` flag in `wrangler.jsonc`). The Gate worker is the only public worker; it owns the login page, the session table, the admin panel, and the dashboard aggregator. Every other module worker is reachable only through Cloudflare **service bindings** — server-to-server calls authenticated by a shared HMAC secret (`GATE_SECRET`) that never leaves Cloudflare's network.

JALARAM-GATE

Login · Auth · Dashboard · Admin
8 D1 DBs (read) · 5 service bindings
Cron: */15, day-end, CRM, low-stock

↓ service bindings (HMAC-signed, `GATE_SECRET`)

FEEDS	PURCHASE	PRODUCTION	LABOUR	LAB
Sales · Orders · Dispatch · WhatsApp	Sauda · Arrival · GRN · Freight	FG batches · Formula · Analysis	Attendance · Payments · Audit	Lab reports · Mobile Link

3.1 Why a single entry point

The Gate pattern buys three things. First, **authentication is centralised**: only the Gate writes to the sessions table, only it sets the HTTP-only session cookie, and only it knows the password hashes. Module workers trust the Gate's signed handoff and never see a password. Second, **blast radius is contained**: if a module worker has a bug, it cannot leak credentials, because it has none. The worst case is a module returning bad data, which the Gate can refuse to display. Third, **cross-cutting features live in one place**: the notification bell, the day-end WhatsApp report, the CRM reminders, and the stock sync job all run as Gate cron triggers, reading across every D1 database through the Gate's eight read bindings.

3.2 Session handoff to module workers

When a logged-in user opens a module (e.g. `/m/feeds/`), the Gate does not proxy the request transparently. Instead, it makes a signed POST to `https://module.internal/gate/session` on the target worker, attaching the `X-Gate-Secret` header. The module worker validates the secret, creates its own local session row (in its own D1 database), and returns a token. The Gate injects that token into the rendered HTML so the frontend can call the module's `/rpc/*` endpoints directly with an `X-Session-Token` header. The module worker is never reachable from the public internet — Cloudflare's service binding is the only path in.

```
// 2-sales-feeds/index.js — Gate handoff endpoint
if (url.pathname === "/gate/session" && request.method === "POST") {
  if (!secretMatches(request.headers.get("X-Gate-Secret"), env.GATE_SECRET))
    return json({ error: "Bad or missing gate secret" }, 403);
  const who = await request.json();
  const token = crypto.randomUUID();
  await env.DB.prepare(
    `INSERT INTO sessions (token, username, role, expires_at) VALUES (?, ?, ?, ?)`
  ).bind(token, who.username, who.role, Date.now() + 8*60*60*1000).run();
  return json({ storageKey: "jf_token", storageValue: token });
}
```

3.3 Worker inventory

Worker	Cron	Lines	Role
jalaram-gate	* /15, 9pm, 9am, 10am IST	9,766	Login, admin, dashboard, CRM, accounts, day-end
jalaram-feeds	hourly	5,120	Sales orders, dispatch, FG stock, WhatsApp
jalaram-purchase	hourly	3,226	Sauda, arrival, GRN, freight, RM stock
jalaram-production	hourly	1,794	FG/SF batches, formula book, analysis
jalaram-labour	hourly	1,073	Attendance, payments, advances, audit
jalaram-lab	half-hourly	544	Lab test reports (login-less via Mobile Link)

4. Authentication & Authorization

The Gate's authentication stack is intentionally boring: **PBKDF2-SHA256** with 100,000 iterations and a 16-byte random salt per password, verified in constant time. Sessions are server-side rows in the jalaram-auth D1 database with an 8-hour TTL, swept every 15 minutes by a cron trigger. The session cookie is HTTP-only, Secure, and SameSite=Lax, so it is not reachable from any client-side JavaScript and is never sent on cross-site POSTs. There is no JWT, no refresh token, no client-side crypto — by design.

4.1 Password hashing

```
async function pbkdf2(password, saltHex, iters) {
  const salt = Uint8Array.from(saltHex.match(/../g).map(h => parseInt(h, 16)));
  const key = await crypto.subtle.importKey("raw", new TextEncoder().encode(password),
    "PBKDF2", false, ["deriveBits"]);
  const bits = await crypto.subtle.deriveBits(
    { name: "PBKDF2", salt, iterations: iters, hash: "SHA-256" },
    key, 256);
  return [...new Uint8Array(bits)].map(b => b.toString(16).padStart(2, "0")).join("")
;
}

async function hashPassword(password) {
  const salt = toHex(crypto.getRandomValues(new Uint8Array(16)));
  return `${salt}$$${ITERS}$$${await pbkdf2(password, salt, ITES)}$`;
}
```

4.2 Per-module RBAC

Beyond login, the Gate enforces a per-module, per-user permission matrix. A users table holds accounts; a modules table holds the list of installed modules; a user_modules junction holds one row per (user, module) grant, with a role ("Admin", "User", or "Viewer") and an allowed-tabs bitmask. When a user opens /m/feeds/, the Gate first checks that a grant exists and that the user's status is "Active"; only then does it issue the signed session handoff. Super-admins bypass the matrix entirely. This keeps the blast radius of a stolen credential small: a salesperson's session token cannot, by construction, open the labour payroll module.

4.3 Mobile Link — B110

A special case is the laboratory technician, who fills sample forms on a mobile device several times a day from the shop floor. Asking them to log in with a username and password each morning was unacceptable. The solution is the **Mobile Link** pattern: an admin generates a long-lived link token (HMAC of a 24-byte random value with GATE_SECRET) and hands the URL to the technician. The link is stored hashed in a device_links table, with a module_code column hard-coded to "lab". When the technician opens the link, the Gate creates a session with scope = "lab" — a session that can open the lab module and nothing else. The link itself is good for 400 days, surviving session expiry. Revoking the link kills all sessions derived from it instantly. This pattern trades a little security surface for a large usability win in a controlled environment.

5. RPC Pattern: One URL, Many Functions

Every Worker in this project exposes the same wire contract: a single POST endpoint at `/rpc/<functionName>` that accepts a JSON array of positional arguments and returns `{ "result": ... }` on success or `{ "error": "..." }` on failure. There is no REST, no GraphQL, no schema. The handler is roughly twenty lines of code per worker. This was a deliberate choice driven by the migration constraint: the legacy Apps Script frontends already spoke `google.script.run`, which is itself a positional-argument RPC. The new backend simply had to speak the same shape.

5.1 The RPC table doubles as the access-control surface

Each worker exports a flat RPC object mapping function names to handlers. Alongside it, three sets encode the access-control policy: `PUBLIC` (callable without authentication — only login), `ADMIN_ONLY` (callable only by super-admins), and `MODULE_RPC` (callable only when the caller's grant includes the relevant module). Adding a new endpoint is therefore a one-line addition to the RPC table and, optionally, one line to an access-control set. The handler itself does not need to re-check permissions. This makes it cheap to add features and difficult to forget the permission gate.

```
var RPC = {
  login, logout, me, changePassword,
  listUsers, createUser, deleteUser, resetPassword, // admin-only
  dashboard, homePending, dailyAudit,             // anyone logged-in
  stockItems, stockAudit, stockLedger, stockAdjustment, // inventory grant
  crmStats, crmLeads, crmSaveLead, crmConvertLead,   // crm grant
  // ... ~150 total endpoints
};
var PUBLIC      = new Set(["login"]);
var ADMIN_ONLY = new Set(["createUser", "deleteUser", "resetPassword", /* ... */]);
var MODULE_RPC = { stockItems: "inventory", crmLeads: "crm", /* ... */ };
```

5.2 Why not REST or GraphQL

A REST or GraphQL layer would have bought us versioned schemas, introspection, and tooling — none of which the frontends needed. The legacy HTML pages were tightly coupled to specific function names and specific argument orders, so a schema would have been pure overhead. The RPC pattern is just the smallest contract that could replace `google.script.run` without touching the frontend. If the team ever grows to the point where frontends are written by people who do not also write the backends, a schema layer can be added later as a thin wrapper — it would not require a rewrite.

6. Database Topology: 8 D1 Databases

Cloudflare D1 is a managed SQLite-at-the-edge product. Each D1 database is independent: it has its own schema, its own migrations, and its own access bindings. The Jalaram Feeds ERP splits its data across eight D1 databases along module boundaries. The Gate worker has **read bindings to all eight** so it can power the cross-module dashboard and the day-end report, but it does not own write access to most of them. Each module worker has write access only to its own database. This keeps the writes flowing through the module that understands the schema, while still letting the Gate aggregate read-only views cheaply.

D1 Database	Owner Worker	Key Tables
jalaram-auth	jalaram-gate	users, sessions, device_links, access_log, daily_audit
jalaram-feeds	jalaram-feeds	orders, dispatches, wa_settings, wa_numbers, audit_log
jalaram-purchase	jalaram-purchase	saudas, arrivals, grn, freights, parties, materials
jalaram-production	jalaram-production	batches, items, formula
jalaram-master	jalaram-gate (admin)	item_master, stock_opening, stock_transfer, stock_adjustment
jalaram-crm	jalaram-gate	crm_leads, crm_notes, crm_payments, crm_settings
jalaram-labour	jalaram-labour	contractors, rates, labour, advances, bank_payments
jalaram-lab	jalaram-lab	lab_reports, sessions

6.1 The stock-sync flow

One of the trickier problems in a multi-database ERP is keeping derived data fresh. The Gate computes a live **stock audit** from raw events (GRNs, production batches, dispatches, transfers, adjustments) and then pushes the result into two downstream databases: `current_stock` in the sales DB (so the sales screen can show available quantity) and `raw_material` plus `pp_bag` in the production DB (so the production planner knows what is on hand). A 15-minute cron trigger on the Gate runs `syncStockToSales` and `syncStockToProduction`, with full logging. If either sync fails, the next run catches up — there is no manual reconciliation step. A small `stock_data_epoch` counter in the master DB is bumped by every stock-mutating RPC, so consumers can cheaply detect "has anything changed since I last read?" without re-scanning every table.

7. Real-World Features Deep Dive

Beyond the architecture, four features in this project demonstrate production-grade engineering at the edge. Each one solves a real operational problem for the business and would not have been possible (or would have been dramatically more expensive) on the previous Apps Script stack.

7.1 WhatsApp Business integration with PDF fallback (B166)

When a salesperson confirms an order, the system sends a WhatsApp message to the customer with the order details. WhatsApp's text API has strict character limits and occasional delivery failures. The original implementation sent only text, and a non-trivial fraction of confirmations never reached the customer. The B166 change added a **PDF fallback**: if the text send returns a failure (or a suspicious "success" that is actually a queued failure), the system generates a PDF of the same content via in-memory PDF bytes (no R2 round-trip) and sends it as a media message. The PDF is rendered in pure JavaScript using a small PDF layout library, so there is no external dependency. The same pattern powers the **pending orders digest** sent to the owner every morning.

```
// 2-sales-feeds/index.js – order confirmation with PDF fallback
async function sendOrderConfirmationWhatsApp(env, head, billTo, orderId, ...) {
  const text = buildOrderText(head, billTo, orderId, newRows, commitmentDate);
  const textRes = await jfWaSendText(env, customerMobile, text);
  if (!textRes.ok) {
    // B166: text failed – fall back to PDF media message
    const pdfBytes = waOrderPdfBytes(order); // in-memory, no R2 round-trip
    const url = await uploadToR2(env, `wa/order-${orderId}.pdf`, pdfBytes);
    return waSendPdf(env, customerMobile, url, "Order confirmation");
  }
  return textRes;
}
```

7.2 Sahayak AI chat assistant (Workers AI)

Sahayak (Hindi for "assistant") is an in-app chat built on Cloudflare Workers AI. It is exposed only to super-admins and lives at /sahayak-chats. The admin types a natural-language question — "show me the orders for Acme Corp this week", "which items are below reorder level at Plant 2", "what dispatches did Rajesh create yesterday" — and the assistant parses the question into one of a fixed set of supported query templates, runs the corresponding RPC, and formats the result as a readable answer in the chat panel. The chat history is persisted in D1. This is a deliberately narrow assistant — it does not free-form generate SQL, it does not have tool use, and it does not execute anything the admin could not have done through the normal UI. The Workers AI binding is used purely as a natural-language-to-template classifier, which keeps the failure modes bounded and the audit trail clean.

7.3 Day End report with Browser Rendering (B128)

Every evening at 9 PM IST, a Gate cron trigger runs the **Day End** pipeline. It collects the day's sales, dispatches, purchases, production batches, and labour entries into a structured HTML report. Then it uses Cloudflare's **Browser Rendering** binding to launch a headless Chromium, navigate to an internal URL that renders the report, take a full-page screenshot, and send that screenshot to the owner's WhatsApp number. The owner thus gets a single image summarising the day's activity without having to open the app. This requires `compatibility_date >= 2026-03-24` in `wrangler.jsonc`; it is one of the newer Workers bindings and is worth highlighting because it dramatically reduces the friction of mobile review for non-technical users.

7.4 Mobile Link for laboratory (B110)

Covered in detail in Section 4.3. The Mobile Link is worth re-listing here as a "real-world feature" because it is the single highest-impact usability change the system has shipped. Before it, the lab technician would log in with a password four to six times per shift. After it, they open a bookmarked URL and the form is on screen. The link is scoped to the lab module only, so even if the phone is lost, the worst case is someone filling fake lab reports — they cannot access payroll, sales, or admin. The link can be revoked from the admin panel in one click, which also kills every session derived from it.

8. Engineering Rigor

The codebase ships with **39 test files** spread across the six workers (22 in Gate, 8 in sales, 4 in purchase, 2 each in production and labour, and 1 in lab). Tests are written in plain Node.js — no Jest, no Mocha, no test framework at all. They use node:sqlite as an in-memory fake for D1 and plain JavaScript functions for mocking fetch. A test file is run by simply invoking it: `node workers/1-gate/print-stamps.test.mjs`. This means tests run in milliseconds, can be debugged with a single `console.log`, and never touch the live database — by construction.

8.1 Test-driven development as a working habit

The project's README.md is explicit about the workflow: "naya kaam TDD se hota hai: pehle test (fail hona chahiye), phir code" — "new work goes through TDD: first the test (which should fail), then the code". Every new feature starts with a small test file that exercises the intended behaviour, watches it fail, then writes the minimum code that flips it green. This is enforced by social convention rather than tooling, but the convention holds because the rest of the codebase is structured this way — writing a feature without a test would feel out of place.

8.2 The "lazy senior dev" philosophy

The project's AGENTS.md file (a deliberate play on the popular "agentic" pattern) opens with a single directive: *"You are a lazy senior developer. Lazy means efficient, not careless. The best code is the code never written."* It then prescribes a seven-rung ladder: first ask if the feature is needed at all (YAGNI), then look for an existing helper, then check the standard library, then a platform feature, then an installed dependency, then ask if it can be one line — and only then write the minimum code that works. The file explicitly forbids abstractions that were not requested, new dependencies, and boilerplate. This is the kind of engineering culture that produces a 22,000-line ERP instead of a 220,000-line one.

QUOTE FROM AGENTS.MD

"Bug fix = root cause, not symptom: a report names a symptom. Grep every caller of the function you touch and fix the shared function once — one guard there is a smaller diff than one per caller, and patching only the path the ticket names leaves a sibling caller still broken."

8.3 B-number changelog convention

Every change in this project is tagged with a B<number> identifier — B110 (Mobile Link), B157 (labour audit log), B166 (WhatsApp PDF fallback), B167 (print attribution). The tag appears in three places: the commit message, the in-code comment that introduces the change, and the README changelog. This makes it trivial to grep for "why was this line added" — the B-number in the comment links directly to the README entry with the context. The numbers are sequential and never reused, even if a change is later reverted. The current highest B-number is B167.

8.4 Coding conventions

Convention	Rule
File encoding	CRLF (Windows line endings) — patches via .cjs scripts to preserve line endings
Comment language	Hinglish — same language the team speaks internally

Convention	Rule
Commit tag	B in commit message, code comment, and README changelog
Data integrity	"Data jaisa hai waisa rehne do" — never silently "fix" a value without asking
New code	TDD — failing test first, then minimum code that passes
Deploy	Only with explicit owner permission; never on a whim

9. Deployment & Operations

Deployment is deliberately manual and gated. The project's main branch is the only branch that ever goes live, and a deploy is triggered only with explicit owner permission. The deploy workflow has six steps, designed to make it impossible to accidentally ship a parked feature or leak a developer's working directory state into production.

9.1 Deploy checklist

- 1. Archive a clean copy** from main. Never deploy from a working folder that has uncommitted changes or stray test files. Use `git archive main workers/<w>` to extract a clean tree into a separate directory.
- 2. Strip test files** from the deploy copy. Test files (*.test.*) are not needed in production and add noise. They are removed before wrangler deploy runs.
- 3. Run deploy guards** for the Gate. Two grep counts must be zero before a Gate deploy is allowed: `loginOtpRequest` (the OTP login feature is parked on a branch and must not be in main) and `MLDB DRY STOCK-50KG` (a stock alias feature is also parked). Both counts must read zero.
- 4. Set the Cloudflare account ID** explicitly. The Gate's `wrangler.jsonc` deliberately omits `account_id` because the project lives in a shared Cloudflare account. Without an explicit export, wrangler may pick the wrong account. Always run `export CLOUDFLARE_ACCOUNT_ID=9945f1b4...` before deploying the Gate.
- 5. Run wrangler deploy** from the worker's clean directory: `node <wrangler.js> deploy`. Watch the output for the deployed URL and version.
- 6. Smoke test** the live URL. Hit `/health` on the Gate and on each module's public URL (only Gate has one). Log in as a known user and verify the dashboard loads.

9.2 Secrets management

Secrets are not stored in `wrangler.jsonc` or in the git repo. They are set via `wrangler secret put <NAME>` and live only in Cloudflare's dashboard. The three secrets in use are `GATE_SECRET` (the HMAC shared between the Gate and module workers), `BRIDGE_TOKEN` (the token that lets the hourly Google Sheet sync script call the snapshot endpoint), and `DAYEND_KEY` (the bearer token for the day-end trigger endpoint, in case the cron job needs to be re-run manually).

9.3 Parked work

Two large features are currently parked on dedicated branches and explicitly not on main: the **OTP login** feature (on `hold/otp-login`) and the **stock alias** feature (on `hold/stock-alias`). Both are waiting for the owner's sign-off before they ship. The deploy guards described above make it impossible to accidentally merge either branch — if a stray reference appears in main, the grep count will be non-zero and the deploy will be blocked.

10. Lessons Learned & What I Would Do Differently

10.1 What worked

The **gas-shim.js proxy pattern** was the single highest-leverage decision of the migration. It let us move module by module without ever flag-daying the application, and it kept the frontends untouched. The cost was a 200-line shim file; the benefit was a zero-downtime migration of an actively used business system. This is the pattern I would reach for first on any future Apps Script migration.

The **service-binding-only private workers** pattern also paid off. The fact that five of six workers have no public URL at all means the attack surface is one worker wide instead of six. It also means module workers do not need their own auth layer — they trust the Gate's signed handoff. This removed an entire class of "did we forget to authenticate this endpoint" bugs.

D1 (SQLite at the edge) was sufficient for the workload. The business has a few thousand orders per month, a few hundred dispatches, a few dozen GRNs. None of this stresses SQLite. The 15-minute stock-sync cron runs in under a second. We never had to introduce a "real" database, and the operational simplicity of D1 (no connection pooling, no replicas, no failover) is a real win.

10.2 What I would do differently

The Gate's `index.js` grew to **9,767 lines**. It works, it is tested, and it is readable, but it has crossed the threshold where a new contributor would struggle to find anything. The next refactor pass should split it into `auth.js`, `dashboard.js`, `stock.js`, `crm.js`, `whatsapp.js`, and `admin.js`, each exporting its RPC table fragment. The pattern is already proven by `helpdesk.js`, `notify.js`, `chat.js`, and `dataedit.js` — it just needs to be applied to the larger file.

The **RPC pattern lacks a schema**. It is convenient and small, but it means there is no machine-readable contract between frontends and backends. A future improvement would be to extract the RPC table into a JSON Schema or TypeScript definition file that both sides can import, without changing the wire protocol. This would let us catch argument-shape mismatches at build time rather than at runtime.

The **CRLF + Hinglish comment conventions** are deliberate — they match the language the team speaks and the editor they use — but they make onboarding harder for non-Hinglish readers. A future team that is more international would benefit from English-only comments and LF line endings. For the current team, the conventions are correct; for the project's long-term portability, they are a trade-off worth noting.

TAKEAWAYS FOR THE NEXT PROJECT

1. A 200-line proxy shim beats a six-month rewrite. Always check if the legacy contract can be preserved.
2. Make private-by-default Workers and route everything through one public entry point. The blast radius shrinks dramatically.
3. D1 is enough for most line-of-business apps. Do not reach for Postgres until you have evidence that SQLite is the bottleneck.
4. Even a "lazy senior dev" codebase eventually needs to be split. Schedule the split before the file hits 10,000 lines, not after.

11. About the Author & Contact

I am **Abhinav Randai**, a software engineer who builds full-stack systems for small and mid-sized businesses. My work tends to live at the intersection of "this needs to actually run in production for years" and "the user is not a developer and should never have to think about the stack". The Jalaram Feeds ERP is a representative project: real business data, real daily users, real operational constraints, and a deliberate choice to keep the architecture as boring as the requirements allow.

I am currently open to opportunities in product engineering, platform engineering, and full-stack roles at companies that value shipping software that lasts. I am particularly interested in teams working on edge computing, developer tooling, and line-of-business systems where the engineering decisions have measurable business impact.

GitHub	<code>github.com/Abhinavrandai</code>
LinkedIn	<code>linkedin.com/in/abhinavrandai</code>
Email	<code>abhinavrandai@gmail.com</code>
Phone	<code>+91-XXXXXXXXXX (replace before sending)</code>

This case study was written in September 2026. The Jalaram Feeds ERP codebase is private; a public mirror with sensitive configuration redacted may be made available on request. The live system runs at `jalaram-gate.jalaramfeeds.workers.dev`, but access is restricted to authorised users of the business.